

Rapport technique — mobileapp-2026

Auteur : CLAPIER Titouan

Date : Avril 2026

Dépôt : mobileapp-2026

Statut : Document de synthèse — toutes applications

Table des matières

1. Contexte et présentation générale
2. Architecture globale du dépôt
3. Application 1 — MyBank (Flutter + Laravel)
 - 3.1 Vue d'ensemble
 - 3.2 Stack technique
 - 3.3 Architecture backend (Laravel)
 - 3.4 API REST — endpoints détaillés
 - 3.5 Base de données — modèle de données
 - 3.6 Application mobile Flutter
 - 3.7 Sécurité
 - 3.8 Démarrage et comptes de test
4. Application 2 — SECONDAPP (React + Vite)
 - 4.1 Vue d'ensemble
 - 4.2 Stack technique
 - 4.3 Architecture frontend
 - 4.4 Module Dashboard
 - 4.5 Module Facturation
 - 4.6 Module Crypto
 - 4.7 NexBank Chain — blockchain de démonstration
 - 4.8 Données mock (Docker + json-server)
 - 4.9 Démarrage
5. Infrastructure et DevOps
6. Comparatif des deux applications
7. Problèmes rencontrés et solutions
8. Perspectives d'évolution
9. Conclusion

1. Contexte et présentation générale

1.1 Genèse du projet

Le projet **mobileapp-2026** est un monorepo centré sur le thème **banque / finance numérique**. Il a été développé en plusieurs phases successives :

1. **Première itération (MyBank)** : application bancaire mobile Flutter associée à une API REST Laravel complète. Cette version a fait l'objet d'une présentation, validant les choix techniques fondamentaux (authentification par token, gestion multi-comptes, virements temps réel).
2. **Deuxième itération (SECONDAPP)** : refonte volontaire depuis zéro après la première présentation. L'objectif était d'aller plus loin sur l'expérience utilisateur (design premium issu de Figma), d'intégrer un module de facturation inspiré d'Odoo, et d'explorer des concepts plus avancés comme une blockchain de démonstration.

1.2 Objectifs pédagogiques

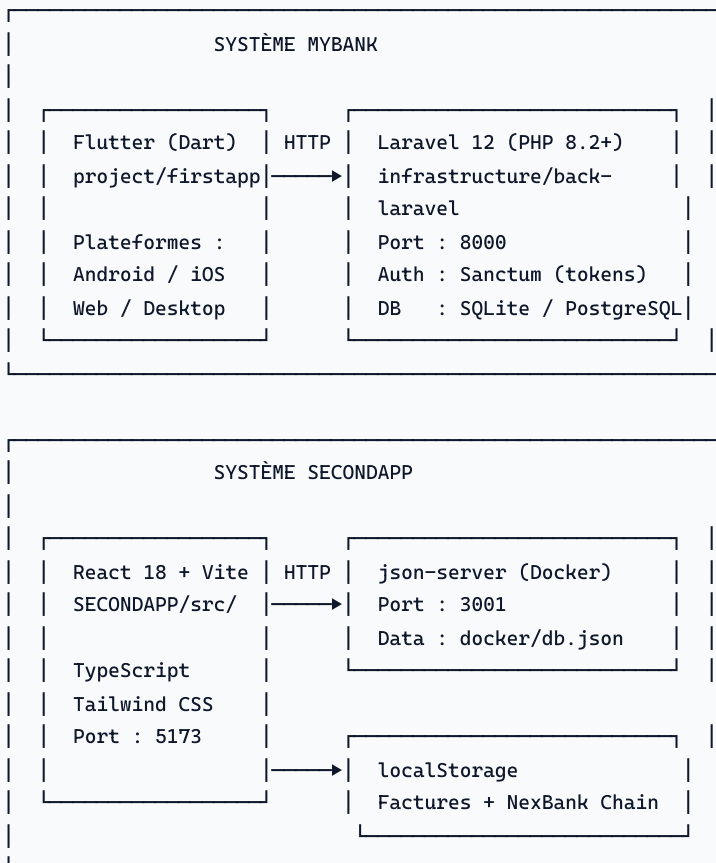
Objectif	Moyen
Maîtrise d'une stack mobile complète	Flutter (Dart) + API REST
Conception d'une API REST sécurisée	Laravel 12 + Sanctum
Développement web moderne	React 18 + TypeScript + Vite
Design system et composants	Tailwind CSS + Radix UI + shadcn
Gestion de données mock	Docker + json-server
Concepts blockchain	SHA-256, Proof of Work, mempool
Module métier type ERP	Facturation inspirée d'Odoo
Containerisation	Docker + Docker Compose

1.3 Structure du monorepo

```
mobileapp-2026/
├─ project/firstapp/      → App mobile Flutter (MyBank)
├─ project/secondapp/    → App Flutter légère (mock)
├─ SECONDAPP/            → App web React (refonte premium)
├─ infrastructure/
│  ├─ back-laravel/      → API REST Laravel (MyBank)
│  ├─ transfer-api/      → Micro-service Node.js
│  ├─ front-next/        → Frontend Next.js (infra)
│  └─ infra/             → Docker Compose (prod)
├─ docs/                 → Documentation et guides
├─ scripts/              → Automatisation PowerShell
└─ README.md
```

2. Architecture globale du dépôt

2.1 Schéma d'ensemble



2.2 Ports et services réseau

Service	Port	Protocole	Description
Laravel API	8000	HTTP	API REST MyBank
SECONDAPP Vite	5173	HTTP	Dev server React
json-server Docker	3001	HTTP	Mock API SECONDAPP
PostgreSQL	5432	TCP	Base de données prod
Next.js (infra)	3000	HTTP	Frontend infrastructure
Nginx proxy	8000	HTTP	Reverse proxy (Docker prod)

3. Application 1 — MyBank (Flutter + Laravel)

3.1 Vue d'ensemble

MyBank est une application bancaire mobile complète et fonctionnelle. Elle permet à plusieurs utilisateurs de gérer leurs comptes, consulter leur historique de transactions, effectuer des virements et interagir avec un chatbot IA. L'application est entièrement connectée à un backend Laravel via une API REST sécurisée.

3.2 Stack technique

Couche	Technologie	Version
App mobile	Flutter / Dart	SDK ^3.4.4
Framework API	Laravel	12.x
Langage backend	PHP	8.2+
Authentification	Laravel Sanctum	4.3
Base de données (dev)	SQLite	—
Base de données (prod)	PostgreSQL	16
ORM	Eloquent (Laravel)	—
État Flutter	Provider	6.x
Persistance locale	flutter_secure_storage	9.x
Préférences	shared_preferences	2.x
Internationalisation	intl	0.19
HTTP client Flutter	http	1.2.x
Tests backend	PHPUnit	11.x
Données de test	Faker (PHP)	1.23

Plateformes Flutter supportées : Android, iOS, Web (Chrome), Windows, Linux, macOS

3.3 Architecture backend (Laravel)

```
infrastructure/back-laravel/
├─ app/
│  ├─ Http/
│  │  └─ Controllers/Api/
│  │     └─ AuthController.php           ← Inscription, connexion, tokens
│  │     └─ AccountController.php        ← Comptes bancaires CRUD
│  │     └─ TransactionController.php     ← Transactions et virements
│  │     └─ PaymentRequestController.php ← Demandes de paiement
│  │     └─ ChatController.php           ← Chatbot IA
│  └─ Models/
│     └─ User.php
│     └─ Account.php
│     └─ Transaction.php
│     └─ Transfer.php
│     └─ PaymentRequest.php
│     └─ Product.php
│  └─ Notifications/
├─ routes/
│  └─ api.php                            ← Définition des routes REST
├─ database/
│  └─ migrations/                        ← 12 fichiers de migration
│  └─ seeders/                          ← Données de test
│  └─ database.sqlite                   ← Fichier SQLite (dev)
└─ config/                             ← Configuration Laravel
```

Middlewares appliqués

- Routes publiques (register, login, forgot/reset password) : **aucun middleware**
- Routes protégées : middleware `auth:sanctum` → vérification du Bearer token à chaque requête

3.4 API REST — endpoints détaillés

Authentification

Méthode	Endpoint	Protection	Description
POST	<code>/api/auth/register</code>	Public	Inscription nouveau compte
POST	<code>/api/auth/login</code>	Public	Connexion → retourne un token Sanctum
POST	<code>/api/auth/logout</code>	Sanctum	Révocation du token courant
POST	<code>/api/auth/forgot-password</code>	Public	Envoi email de réinitialisation
POST	<code>/api/auth/reset-password</code>	Public	Réinitialisation avec le token email
POST	<code>/api/auth/change-password</code>	Sanctum	Changement du mot de passe authentifié

Comptes bancaires

Méthode	Endpoint	Description
GET	/api/accounts	Liste tous les comptes de l'utilisateur
GET	/api/accounts/{id}	Détail d'un compte (solde, type)
POST	/api/accounts	Créer un nouveau compte
POST	/api/accounts/{id}/debit	Débiter un compte
POST	/api/accounts/{id}/credit	Créditer un compte
DELETE	/api/accounts/{id}	Supprimer un compte
GET	/api/beneficiaries	Liste des bénéficiaires disponibles

Transactions

Méthode	Endpoint	Description
GET	/api/transactions	Historique complet de l'utilisateur
POST	/api/transactions/transfer	Effectuer un virement
GET	/api/accounts/{id}/transactions	Transactions d'un compte précis

Demandes de paiement

Méthode	Endpoint	Description
GET	/api/payment-requests	Liste des demandes
POST	/api/payment-requests	Créer une demande
POST	/api/payment-requests/{id}/accept	Accepter une demande
POST	/api/payment-requests/{id}/decline	Refuser une demande

Chatbot IA

Méthode	Endpoint	Description
POST	/api/chat	Envoyer un message au chatbot

3.5 Base de données — modèle de données

Schéma des tables principales

```
users
├─ id (PK)
├─ first_name, last_name
├─ email (unique)
├─ password (bcrypt)
├─ phone, address
└─ avatar_url

accounts
├─ id (PK)
├─ user_id (FK → users)
├─ account_number (unique, généré)
├─ account_type (checking | savings)
└─ balance (decimal)

transactions
├─ id (PK)
├─ from_account_id (FK → accounts, nullable)
├─ to_account_id (FK → accounts, nullable)
├─ amount (decimal)
├─ type (debit | credit | transfer)
└─ description

transfers
├─ id (PK)
├─ source_account_id (FK → accounts)
├─ destination_account_id (FK → accounts)
├─ amount (decimal)
└─ status (pending | completed | failed)

payment_requests
├─ id (PK)
├─ requester_id (FK → users)
├─ target_id (FK → users)
├─ amount (decimal)
└─ status (pending | accepted | declined)

personal_access_tokens (Laravel Sanctum)
├─ tokenable_type, tokenable_id
├─ name, token (hash SHA-256)
└─ expires_at
```

Données de test (Seeder)

Après `php artisan migrate:fresh --seed` :

Utilisateur	Email	Mot de passe
Jean Dupont	jean.dupont@example.com	password123
Marie Martin	marie.martin@example.com	password123
Pierre Bernard	pierre.bernard@example.com	password123
Sophie Lefebvre	sophie.lefebvre@example.com	password123

Chaque utilisateur possède plusieurs comptes (courant + épargne) avec des transactions pré-existantes.

3.6 Application mobile Flutter

Structure de l'application

```

project/firstapp/lib/
├─ main.dart           ← Point d'entrée, initialisation Provider
├─ config/             ← Configuration API (base URL, headers)
├─ models/             ← Modèles Dart (User, Account, Transaction...)
├─ screens/            ← 17 écrans de l'application
│   ├─ login_screen.dart
│   ├─ dashboard_screen.dart
│   ├─ account_detail_screen.dart
│   ├─ transfer_screen.dart
│   ├─ transactions_screen.dart
│   ├─ payment_request_screen.dart
│   ├─ chat_screen.dart
│   └─ profile_screen.dart
├─ services/           ← Services HTTP (appels API)
└─ theme/              ← Thème visuel Material Design

```

Gestion de l'état

L'application utilise **Provider** comme solution de gestion d'état :

- **AuthProvider** : état de connexion, token Sanctum, données utilisateur
- **AccountProvider** : liste des comptes, soldes actuels
- **TransactionProvider** : historique des transactions
- Chaque provider communique avec son service HTTP correspondant

Sécurité Flutter

- **Token Sanctum** : stocké dans `flutter_secure_storage` (keychain iOS / keystore Android)
- Jamais dans `SharedPreferences` (stockage non chiffré)
- Toutes les requêtes authentifiées incluent `Authorization: Bearer {token}`
- Déconnexion = révocation côté Laravel + suppression locale du token

Configuration réseau

```
// En développement local (Web / Desktop) :  
const String baseUrl = 'http://127.0.0.1:8000/api';  
  
// Sur émulateur Android :  
const String baseUrl = 'http://10.0.2.2:8000/api';
```

3.7 Sécurité

Mécanisme	Implémentation
Hachage mots de passe	Bcrypt, 12 rounds (configurable dans <code>.env</code>)
Authentification API	Laravel Sanctum — Bearer tokens
Stockage token mobile	flutter_secure_storage (chiffré OS)
Validation des données	Laravel Form Requests (côté serveur)
Protection CSRF	N/A (API stateless, token-based)
Expiration tokens	Configurable via Sanctum

3.8 Démarrage et commandes

Backend Laravel

```
cd infrastructure\back-laravel  
  
# Installation des dépendances  
composer install  
  
# Configuration  
copy .env.example .env  
php artisan key:generate  
  
# Base de données et données de test  
php artisan migrate:fresh --seed  
  
# Démarrage du serveur  
php artisan serve --host=127.0.0.1 --port=8000
```

Application Flutter

```
cd project\firstapp

# Récupération des dépendances
flutter pub get

# Lancement en mode Web (Chrome)
flutter run -d chrome

# Lancement sur émulateur Android
flutter run

# Lancement sur le bureau Windows
flutter run -d windows
```

Scripts automatisés (PowerShell)

```
# Depuis la racine du dépôt :

# MyBank complet (Laravel + Flutter Web)
.\scripts\run_web.ps1

# MyBank sur émulateur Android
.\scripts\run_emulator.ps1

# Réinitialisation base de données
.\scripts\reset_db.ps1
```

4. Application 2 — SECONDAPP (React + Vite)

4.1 Vue d'ensemble

SECONDAPP est la deuxième application du projet, développée depuis zéro après la première présentation de MyBank. Elle est orientée **interface premium** (design issu d'une maquette Figma), fonctionne entièrement dans le navigateur, et utilise Docker pour servir ses données de démonstration.

Ses trois piliers fonctionnels sont :

1. **Dashboard bancaire** avec graphiques et analyses financières
2. **Module de facturation** inspiré d'Odoo
3. **NexBank Chain** — blockchain de démonstration locale

4.2 Stack technique

Catégorie	Bibliothèque	Version
Framework	React	18.3.1
Build tool	Vite	6.4.1
Langage	TypeScript	—
Styles	Tailwind CSS	4.1.12
Composants UI	Radix UI (shadcn)	—
Composants Material	MUI + MUI Icons	7.3.5
Icônes	Lucide React	0.487.0
Animations	Motion (ex-Framer)	12.x
Graphiques	Recharts	2.15.2
Routing	React Router	7.13.0
Formulaires	React Hook Form	7.55.0
Dates	date-fns	3.6.0
Drag & Drop	react-dnd	16.x
Toast/notifications	Sonner	2.x
Carrousel	embla-carousel-react	8.x
OTP	input-otp	1.x
Confettis	canvas-confetti	1.9.4
Mock API	clue/json-server (Docker)	0.12
Persistance locale	localStorage	—

4.3 Architecture frontend

```
SECONDAPP/src/app/
├─ App.tsx                ← Composant racine, providers
├─ routes.tsx             ← 12 routes définies (React Router)
├─ contexts/
│   ├─ AuthContext.tsx    ← Contexte d'authentification
│   └─ ThemeContext.tsx   ← Thème clair/sombre
├─ components/
│   ├─ Dashboard.tsx      ← Tableau de bord principal
│   ├─ Transfers.tsx      ← Virements
│   ├─ Cards.tsx          ← Cartes bancaires
│   ├─ Investments.tsx     ← Investissements
│   ├─ Analytics.tsx      ← Analytiques financières
│   ├─ Crypto.tsx         ← Cryptomonnaies
│   ├─ Invoicing.tsx      ← Facturation (module principal)
│   ├─ Blockchain.tsx     ← NexBank Chain (blockchain)
│   ├─ Login.tsx / SignUp.tsx ← Authentification
│   ├─ Profile.tsx / Security.tsx ← Profil et sécurité
│   ├─ charts/            ← Composants Recharts dédiés
│   └─ effects/           ← Effets visuels premium
│       ├─ GlowCard.tsx
│       └─ HolographicCard.tsx
│           └─ AnimatedBorder.tsx
├─ modals/                ← Boîtes de dialogue
├─ ui/                    ← Composants UI réutilisables (shadcn)
├─ figma/                 ← Composants issus de la maquette Figma
└─ services/
    ├─ dashboardApi.ts    ← Appels API dashboard
    ├─ invoicingApi.ts    ← Appels API facturation
    ├─ invoicingCalculations.ts ← Calculs HT/TVA/TTC
    ├─ invoicingPersistence.ts ← Lecture/écriture localStorage
    └─ nexbank-chain/
        ├─ engine.ts      ← Moteur blockchain (SHA-256, PoW)
        ├─ types.ts       ← Types TypeScript (Block, Chain...)
        ├─ storage.ts     ← Persistance localStorage
        ├─ walletSync.ts  ← Sync avec module Crypto
        └─ index.ts       ← Export public
```

Routes de l'application

Route	Composant	Description
/login	Login	Page de connexion
/signup	SignUp	Inscription
/	Dashboard	Tableau de bord (protégé)
/transfers	Transfers	Virements
/crypto	Crypto	Cryptomonnaies
/investments	Investments	Investissements
/cards	Cards	Cartes bancaires
/analytics	Analytics	Analytiques
/security	Security	Sécurité et 2FA
/profile	Profile	Profil utilisateur
/invoices	Invoicing	Facturation
/blockchain	Blockchain	NexBank Chain

Flux de données

```

Navigateur (React)
|
▼ fetch("/json-api/*")
Proxy Vite (vite.config.ts)
|
▼ http://127.0.0.1:3001/*
Docker json-server
|
▼ lecture fichier
docker/db.json

```

Pour les factures et la blockchain, la persistance passe par le **localStorage** du navigateur.

4.4 Module Dashboard

Le dashboard (/) est le point d'entrée principal de l'application après connexion.

Composants

- **Solde** : affichage du solde courant avec option masquer/afficher
- **Graphique d'évolution** : courbe Recharts alimentée par `balanceData` (janvier à juin)
- **Transactions récentes** : liste avec catégorie, montant et horodatage

- **Insights** : cartes informatives (taux d'épargne, alertes dépenses, croissance)

Résilience

Si Docker est arrêté ou si json-server ne répond pas, le dashboard affiche un message explicite plutôt qu'une erreur silencieuse. L'application reste navigable.

4.5 Module Facturation

Le module de facturation (`/invoices`) implémente les concepts fondamentaux du module comptable d'Odoo.

Modèle de données

```
interface Partner {
  id: number;
  name: string;
  vat: string;           // Numéro de TVA intracommunautaire
  email: string;
  phone: string;
  address: string;
  city: string;
  country: string;
}

interface InvoiceLine {
  id: number;
  description: string;
  quantity: number;
  unit_price: number;
  tax_rate: number;      // En pourcentage (ex: 20 pour 20%)
}

interface Invoice {
  id: number;
  partner_id: number;
  state: 'draft' | 'posted' | 'paid';
  date: string;
  lines: InvoiceLine[];
  amount_untaxed: number; // Total HT
  amount_tax: number;     // Total TVA
  amount_total: number;   // Total TTC
}
```

Fonctionnement par état

État	Affichage	Actions disponibles
<code>draft</code>	Badge « Brouillon »	Modifier, supprimer (si créé en local), annuler modifications
<code>posted</code>	Badge « À encaisser »	Lecture seule
<code>paid</code>	Badge « Payée »	Lecture seule

Persistence localStorage

- Clé utilisée : `secondapp-invoicing-drafts-v1`
- **Fusion au chargement** : les brouillons locaux écrasent les entrées de même `id` venant du JSON
- **Identifiants négatifs** pour les factures créées dans l'app (évite les collisions avec le JSON)
- Possibilité de « Reprendre la version serveur » pour un brouillon `id` positif

Calculs automatiques

Le service `invoicingCalculations.ts` recalcule en temps réel :

- Sous-total HT par ligne : `quantité × prix_unitaire`
- TVA par ligne : `sous-total × (taux_tva / 100)`
- Total HT global : somme des sous-totaux
- Total TVA global : somme des TVA par ligne
- Total TTC : `HT + TVA`

4.6 Module Crypto

L'écran Crypto (`/crypto`) présente une interface de trading avec :

- Liste des cryptomonnaies disponibles avec prix simulés
- Graphiques d'évolution de cours (Recharts)
- Formulaire d'achat
- **Intégration NexBank Chain** : chaque achat enregistre automatiquement une transaction de type `wallet` dans le mempool de la blockchain, via `walletSync.ts`

4.7 NexBank Chain — blockchain de démonstration

NexBank Chain est une implémentation pédagogique complète d'une blockchain fonctionnelle, entièrement locale dans le navigateur.

Concepts implémentés

Concept	Implémentation	Fichier
Hachage	SHA-256 via Web Crypto API	<code>engine.ts</code>
Bloc	<code>{ index, timestamp, data, previousHash, hash, nonce }</code>	<code>types.ts</code>
Bloc genèse	Premier bloc fixe, <code>previousHash = "0"</code>	<code>engine.ts</code>
Proof of Work	<code>hash.startsWith("0".repeat(difficulty))</code>	<code>engine.ts</code>
Difficulté	Configurable (défaut N=2 zéros hexadécimaux)	<code>engine.ts</code>
Mempool	File d'attente <code>Transaction[]</code> avant minage	<code>engine.ts</code>
Validation	Vérification <code>previousHash</code> + hash PoW chaîne entière	<code>engine.ts</code>
Persistance	<code>localStorage</code> — clé <code>secondapp-nexbank-chain-v1</code>	<code>storage.ts</code>
Sync externe	Transactions Crypto → mempool	<code>walletSync.ts</code>

Flux de traitement complet

```

1. Utilisateur ajoute une transaction (ou achat Crypto)
  ↓
2. Transaction enregistrée dans le Mempool
   { from, to, amount, timestamp, type }
  ↓
3. Clic "Miner un bloc"
  ↓
4. Algorithme PoW :
   nonce = 0
   while (!hash.startsWith("0".repeat(difficulty))) {
     nonce++
     hash = SHA256(index + data + previousHash + nonce)
   }
  ↓
5. Bloc créé et ajouté à la chaîne
   { index, timestamp, transactions: [...mempool], previousHash, hash, nonce }
  ↓
6. Mempool vidé
  ↓
7. "Vérifier la chaîne" :
   - Bloc genèse = bloc[0]
   - Pour chaque bloc i > 0 :
     bloc[i].previousHash == bloc[i-1].hash ?
     recalcul SHA256 == bloc[i].hash ?
     hash commence par N zéros ?

```

Interface utilisateur

- Visualisation de la chaîne complète (liste de blocs avec leurs transactions)
- Formulaire d'ajout de transaction au mempool

- Bouton "Miner" avec animation de progression
- Bouton "Vérifier l'intégrité" avec résultat visuel
- Statistiques : nombre de blocs, transactions totales, dernier hash
- Persistance automatique après chaque minage

4.8 Données mock (Docker + json-server)

Docker Compose

```
services:
  json-server:
    image: clue/json-server:latest
    container_name: secondapp-db
    ports:
      - "3001:80"
    volumes:
      - ./docker/db.json:/data/db.json:ro # Lecture seule
    restart: unless-stopped
```

Structure de db.json

```

{
  "dashboard": {
    "balanceData": [
      { "month": "Jan", "balance": 4200 },
      { "month": "Fév", "balance": 4850 },
      { "month": "Mar", "balance": 4100 },
      { "month": "Avr", "balance": 5200 },
      { "month": "Mai", "balance": 4900 },
      { "month": "Juin", "balance": 5650 }
    ],
    "transactions": [
      { "id": 1, "label": "Loyer", "amount": -950, "category": "Logement", ... },
      ...
    ],
    "insights": [
      { "type": "savings_rate", "value": 18, "label": "Taux d'épargne ce mois" },
      ...
    ],
    "invoicing": {
      "partners": [
        { "id": 1, "name": "ACME Corp", "vat": "FR123456789", ... },
        ...
      ],
      "invoices": [
        { "id": 1, "state": "posted", "partner_id": 1, "lines": [ ... ], ... },
        ...
      ]
    }
  }
}

```

Note technique : Avec `clue/json-server` (version 0.12), les routes `/partners` et `/invoices` en racine JSON retournaient des 404. La solution adoptée a été d'imbriquer toutes les données sous la clé `dashboard` et d'effectuer un seul `GET /dashboard`.

Proxy Vite (vite.config.ts)

```

server: {
  proxy: {
    '/json-api': {
      target: 'http://127.0.0.1:3001',
      changeOrigin: true,
      rewrite: (path) => path.replace(/^\/json-api/, '')
    }
  }
}

```

Le proxy évite les erreurs **CORS** en développement : le navigateur envoie ses requêtes au serveur Vite (même origine), qui les transfère au container Docker.

4.9 Démarrage

```
cd SECONDAPP

# Installation des dépendances
npm install

# Démarrage du container json-server
npm run docker:up

# Lancement du serveur de développement
npm run dev
# → http://localhost:5173

# Arrêt du container
npm run docker:down
```

5. Infrastructure et DevOps

5.1 Docker Compose — Stack de production complète

Le fichier `infrastructure/infra/docker-compose.yml` définit une stack complète pour le déploiement de MyBank en environnement conteneurisé :

```
infrastructure/infra/
├─ docker-compose.yml
└─ nginx.conf
```

Services

Service	Image	Port	Rôle
db	postgres:16	5432	Base de données PostgreSQL
backend	Dockerfile local	8001	API Laravel (PHP-FPM)
frontend	Dockerfile local	3000	Frontend Next.js
nginx	nginx:alpine	8000	Reverse proxy (point d'entrée)

Network : `app_network` (driver bridge)

Volumes : `db_data` (persistance PostgreSQL)

Commandes Docker

```
cd infrastructure\infra

# Démarrage complet
docker compose up -d

# Accès principal via Nginx
# http://localhost:8000

# Arrêt
docker compose down

# Logs
docker compose logs -f backend
```

5.2 Micro-service Transfer API

Le dossier `infrastructure/transfer-api/` contient un micro-service Node.js minimal basé sur Express :

Dépendance	Rôle
express	Serveur HTTP
cors	Gestion des headers CORS
uuid	Génération d'identifiants uniques

Ce service sert de démonstration d'architecture micro-services pour les transferts, indépendamment du backend Laravel principal.

5.3 Scripts PowerShell (Windows)

Script	Dossier cible	Description
<code>run_web.ps1</code>	<code>scripts/</code>	Démarre Laravel (port 8000) + Flutter Web (Chrome)
<code>run_emulator.ps1</code>	<code>scripts/</code>	Démarre Laravel + émulateur Android + Flutter
<code>reset_db.ps1</code>	<code>scripts/</code>	Réinitialise la base de données SQLite + reseed
<code>test_transfer.ps1</code>	<code>scripts/</code>	Teste l'API de transfert avec des requêtes HTTP

6. Comparatif des deux applications

Critère	MyBank	SECONDAPP
Type	Application mobile + API	Application web
Frontend	Flutter (Dart)	React 18 (TypeScript)
Backend	Laravel 12 (PHP)	json-server (Docker, mock)
Base de données	SQLite / PostgreSQL (réelle)	Fichier JSON statique
Authentification	Réelle (tokens Sanctum)	Simulée (UI uniquement)
Persistance	Base de données relationnelle	localStorage + JSON Docker
Plateformes	Android, iOS, Web, Desktop	Navigateur uniquement
Données	Réelles, créées par l'utilisateur	Mock, pré-définies en JSON
Graphiques	Non (v1)	Recharts (courbes, secteurs)
Facturation	Non	Oui (inspiré Odoo)
Blockchain	Non	Oui (NexBank Chain, démo)
Crypto	Non	Interface démo
Design	Material Design (Flutter)	Premium, Figma-based
Animations	Flutter transitions	Motion (ex-Framer)
Conteneurisation	Docker Compose (prod)	Docker (json-server seul)
Tests	PHPUnit (Laravel)	Non implémentés
Complexité codebase	Moyenne	Élevée (20+ composants)

Points forts de MyBank

- **Backend complet et fonctionnel** : toute la logique métier bancaire est implémentée
- **Sécurité robuste** : authentification réelle, hachage, stockage sécurisé
- **Multi-plateforme** : une seule base de code Flutter pour toutes les plateformes
- **Données persistantes** : les actions ont un effet réel en base de données

Points forts de SECONDAPP

- **UI premium** : design soigné, effets visuels, animations fluides
- **Autonomie** : fonctionne sans backend propre grâce à Docker + json-server
- **Fonctionnalités avancées** : blockchain, facturation, analytics
- **Modern stack** : React 18, Vite 6, Tailwind 4, TypeScript strict

7. Problèmes rencontrés et solutions

MyBank

Problème	Cause	Solution retenue
CORS sur émulateur Android	L'adresse <code>127.0.0.1</code> ne résout pas vers l'hôte depuis l'émulateur	Utiliser <code>10.0.2.2</code> pour l'adresse de l'hôte Android
Symlinks Flutter sous Windows	Windows demande des privilèges pour créer des jonctions	Activer le mode développeur ou lancer en administrateur
SQLite en multi-connexions	Limitations SQLite pour les tests concurrents	Utiliser PostgreSQL pour les environnements de test

SECONDAPP

Problème	Cause	Solution retenue
CORS entre Vite (5173) et json-server (3001)	Origines différentes bloquées par le navigateur	Proxy Vite <code>/json-api</code> → <code>http://127.0.0.1:3001</code>
Données absentes sur <code>/partners</code> et <code>/invoices</code>	json-server 0.12 (image clue) — comportement différent des versions récentes	Imbrication sous <code>dashboard</code> → un seul <code>GET /dashboard</code>
Impossibilité d'écrire dans <code>db.json</code>	Volume Docker monté en lecture seule (<code>:ro</code>)	Persistance des modifications dans <code>localStorage</code>
Perte de données factures au rechargement	localStorage vide si cache effacé	Fusion au chargement : JSON serveur + localStorage

8. Perspectives d'évolution

Pour MyBank

- **Tests automatisés** : extension de la suite PHPUnit, ajout de tests d'intégration
- **Interface web Next.js** : le dossier `front-next` offre une base pour une version web de MyBank
- **Notifications push** : Laravel Notifications (email, SMS) pour les virements reçus
- **Authentification 2FA** : intégration d'un second facteur (TOTP, SMS)
- **Export de relevés** : génération de PDF de relevés bancaires

Pour SECONDAPP

- **Backend réel** : remplacer json-server par une API Laravel ou Node.js avec PostgreSQL
- **Authentification** : implémenter une vraie authentification (JWT ou session)
- **NexBank Chain** : connecter à un vrai réseau (testnets Ethereum, Polygon)
- **Facturation** : persistance serveur, génération PDF de factures, envoi par email
- **Tests** : Vitest pour les services, Playwright pour les tests E2E
- **Internationalisation** : i18n (français / anglais)

Infrastructure commune

- **CI/CD** : GitHub Actions pour tests, build et déploiement automatisés
- **Monitoring** : intégration d'observabilité (Grafana, Sentry)
- **Hébergement** : Docker Compose de production vers Kubernetes ou un PaaS

9. Conclusion

Le projet **mobileapp-2026** démontre la maîtrise d'un spectre technique large et cohérent autour du domaine bancaire :

MyBank illustre comment concevoir une **application mobile professionnelle** avec une architecture backend solide. La combinaison Flutter + Laravel respecte les bonnes pratiques de sécurité (tokens Sanctum, bcrypt, stockage sécurisé) et offre une expérience cross-plateforme native.

SECONDAPP démontre la capacité à construire une **interface web premium** avec les technologies modernes React / TypeScript, tout en intégrant des fonctionnalités avancées comme une blockchain fonctionnelle (NexBank Chain) et un module de facturation inspiré d'un ERP réel (Odoo). La résolution des contraintes techniques (CORS, json-server 0.12, volume Docker lecture seule) témoigne d'une capacité d'adaptation et de prise de décision d'architecture.

L'ensemble du projet est **documenté, reproductible et présentable** : guides débutants, README détaillés, scripts d'automatisation, et ce présent rapport qui en constitue la synthèse complète.

mobileapp-2026 · CLAPIER Titouan · Avril 2026
